



Drone Choreo "SkyOrchestrator"

Software-Entwicklungspraktikum 2025

Mohammed Alhalabi, Luan Hyseni, Mohamed Sabri Jlizi, Mohammad Khatib,
Lucas Neidahl, Marvin Voigt

Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik

23. Mai 2025

Agenda

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

Ausblick

Inhalt

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

Ausblick

Motivation

Warum SkyOrchestrator?

- ▶ Visuelle Effekte sind integraler Bestandteil der modernen Welt
- ▶ bietet alternative zu Feuerwerk/Lichtshows
 - minimale Umweltauswirkung
 - reduzierten Sicherheitsrisiken
 - erhöhte Flexibilität und kreative Ausdrucksmöglichkeit

Inhalt

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

Ausblick

Zielsetzung & Hauptfunktionen

Was soll die Software können?

- ▶ Flugbahnen & Formationen definieren
- ▶ Licht, Farben & Zeitpunkte steuern
- ▶ Anzahl der Dornen, sowie deren Startpositionen festlegen
- ▶ Pfade im 3D-Raum definieren
- ▶ Echtzeit-Simulation
- ▶ Zeitliche Steuerung und Ablaufplanung
- ▶ (*Optional*) Musik, Export, Kollisionserkennung und Optimierung der Flugbahnen

Inhalt

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

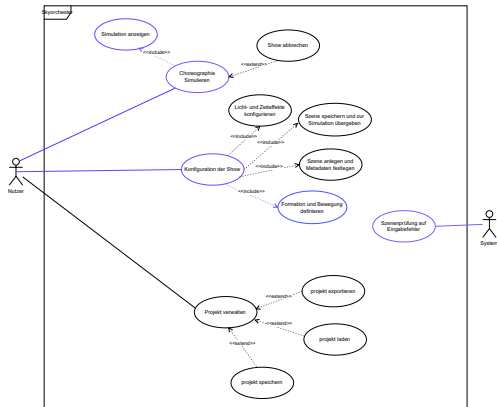
Ausblick

Aktueller Stand

Was haben wir bis jetzt Umgesetzt

- ▶ Use Case Diagramm
- ▶ Prototype showcase

Use Case Diagramm



Prototype Showcase

Inhalt

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

Ausblick

Bisherige Umsetzung & Technik

Was ist bisher entstanden?

- ▶ PyBullet Prototype
- ▶ Erste Funktionsprototypen
- ▶ sehr simple GUI

Einblick:

```
85 def interpolate(current, target, step_size):
86     current = np.array(current)
87     target = np.array(target)
88     direction = target - current
89     dist = np.linalg.norm(direction)
90     if dist < step_size:
91         return target
92     return current + direction / dist * step_size
```

Abbildung: Funktion interpolate

```
66 def kreisformation(n, radius, height, center=(0, 0)):  
67     positions = []  
68     angles = np.linspace(0, 2 * np.pi, n, endpoint=False)  
69     for angle in angles:  
70         x = center[0] + radius * np.cos(angle)  
71         y = center[1] + radius * np.sin(angle)  
72         z = height  
73         positions.append((x, z, y+4.3))  
74     return positions
```

Abbildung: Funktion Kreis

```
112 def F_formation_vertical(n, height, d):
113     segments = [(0, -1), (0, 1),
114                 (0, 1), (1.5, 1),
115                 (0, 0), (1, 0)]
116     seg_counts = [n//3, n//3, n - 2*(n//3)]
117     pos = []
118     for (start, end), cnt in zip(segments, seg_counts):
119         y0, z0 = start; y1, z1 = end
120         for i in range(cnt):
121             t = i / (cnt - 1) if cnt > 1 else 0
122             y = y0 + (y1 - y0) * t
123             z = z0 + (z1 - z0) * t
124             pos.append((0, y + d, z + height + 1))
125     return pos
```

Abbildung: Funktion Buchstabe F

Inhalt

Motivation

Zielsetzung & Hauptfunktionen

Aktueller Stand

Bisherige Umsetzung & Technik

Ausblick

Ausblick

Wie geht's weiter?

- ▶ GUI ausbauen und interaktiv umsetzen
- ▶ bestehende Funktionen optimieren
- ▶ Funktionen erweitern (z.B. weitere Formen)
- ▶ Bewegungsanimationen & Flugverlauf
- ▶ Kamerasteuerung optimieren
- ▶ Fehlerbehandlung ausbauen
- ▶ Lichtdarstellung
- ▶ Export & evtl. Musik-Synchronisierung

Vielen Dank für Ihre Aufmerksamkeit!
Haben Sie Fragen?



Abbildung: GUI Mockup

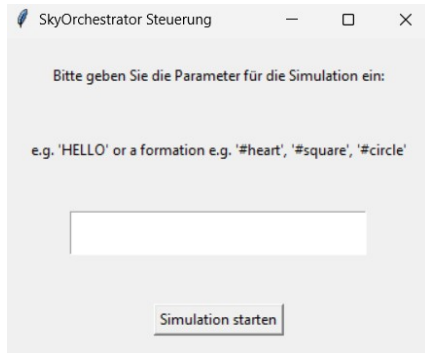


Abbildung: "GUI"

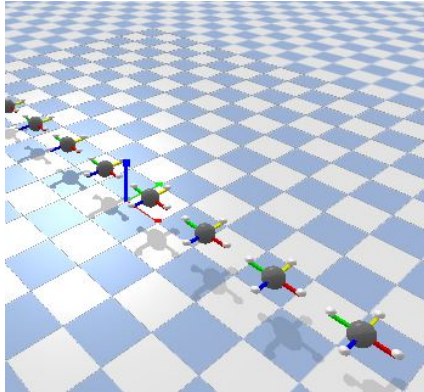


Abbildung: Prototype: HELLO

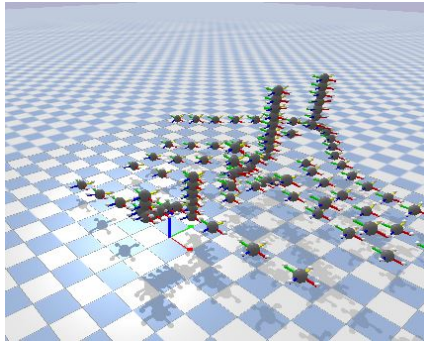


Abbildung: Prototype: HELLO

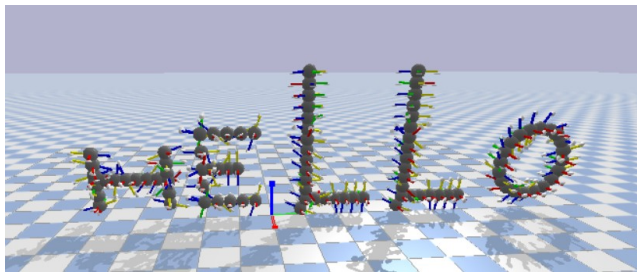


Abbildung: Prototype: HELLO

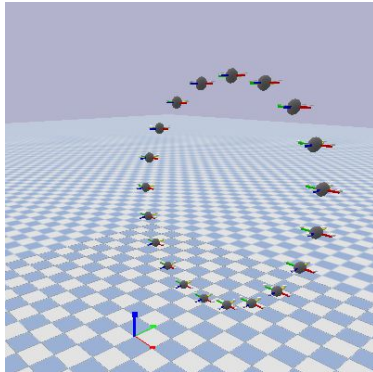


Abbildung: Prototype: Kreis